

PRAKTIKUM SISTEM OPERASI

MODUL IX

Syscall



Oleh:

Haza Zaidan Zidna Fann

2311104056

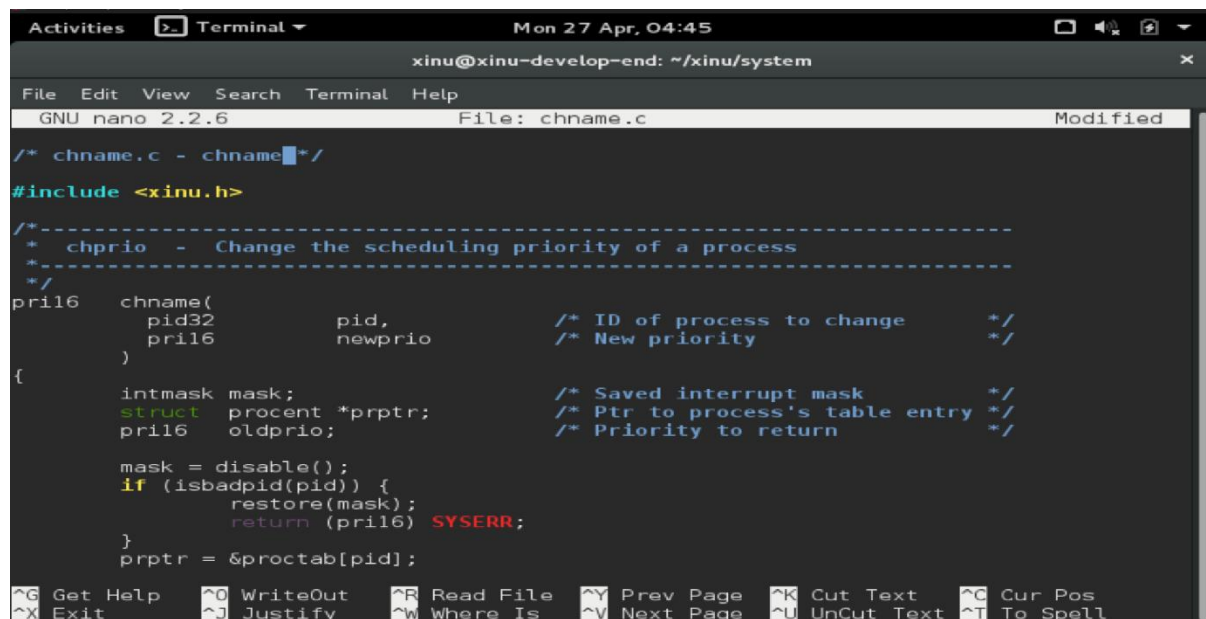
PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO UNIVERSITAS TELKOM

2026

JURNAL

1.



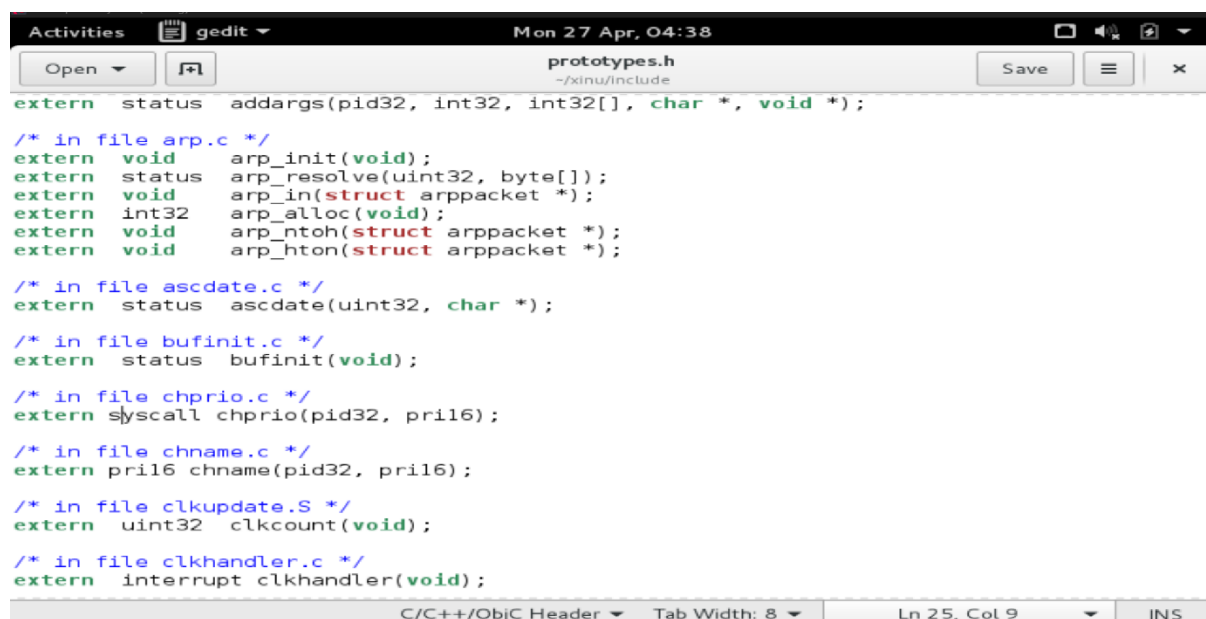
The screenshot shows a terminal window with the title bar "xinu@xinu-develop-end: ~/xinu/system". The window contains the nano text editor editing the file "chname.c". The editor's menu bar includes "File", "Edit", "View", "Search", "Terminal", and "Help". The status bar at the bottom shows "GNU nano 2.2.6" and "File: chname.c". The code in the editor is as follows:

```
/* chname.c - chname */
#include <xinu.h>

/*-----
 * chprio - Change the scheduling priority of a process
 *-----
 */
pril6 chname(
    pid32 pid,          /* ID of process to change */
    pril6 newprio       /* New priority */
)
{
    intmask mask;
    struct proctent *prptr; /* Ptr to process's table entry */
    pril6 oldprio;         /* Priority to return */

    mask = disable();
    if (isbadpid(pid)) {
        restore(mask);
        return (pril6) SYSERR;
    }
    prptr = &proctab[pid];
```

The bottom status bar of the nano editor shows various keyboard shortcuts: ^G Get Help, ^O WriteOut, ^R Read File, ^Y Prev Page, ^K Cut Text, ^C Cur Pos, ^X Exit, ^J Justify, ^W Where Is, ^V Next Page, ^U UnCut Text, and ^T To Spell.



The screenshot shows a gedit text editor window with the title bar "Mon 27 Apr, 04:38". The window contains the file "prototypes.h" located at "~/xinu/include". The editor's menu bar includes "Open", "Save", and a hamburger menu icon. The code in the editor is as follows:

```
extern status addargs(pid32, int32, int32[], char *, void *);

/* in file arp.c */
extern void arp_init(void);
extern status arp_resolve(uint32, byte[]);
extern void arp_in(struct arppacket *);
extern int32 arp_alloc(void);
extern void arp_ntoh(struct arppacket *);
extern void arp_hton(struct arppacket *);

/* in file ascdatetime.c */
extern status ascdatetime(uint32, char *);

/* in file bufinit.c */
extern status bufinit(void);

/* in file chprio.c */
extern syscall chprio(pid32, pril6);

/* in file chname.c */
extern pril6 chname(pid32, pril6);

/* in file clkupdate.S */
extern uint32 clkcount(void);

/* in file clkhandler.c */
extern interrupt clkhandler(void);
```

The bottom status bar of the gedit editor shows "C/C++/ObjC Header", "Tab Width: 8", "Ln 25, Col 9", and "INS".

```
Activities Terminal Mon 27 Apr, 02:43
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Help

-- -- ---- - - ----
-----

elcome to Xinu!

sh $ ps
id Name State Prio Ppid Stack Base Stack Ptr Stack Size
-----
0 prnull ready 0 0 0x005FDFFC 0x00FFFE4 8192
1 ipout wait 500 0 0x005FBFFC 0x005FBF30 8192
2 netin wait 500 0 0x005F9FFC 0x005F9ED0 8192
4 Main process recv 20 3 0x005E7FFC 0x005E7F44 65536
5 shell recv 50 4 0x005D7FFC 0x005D79AC 8192
6 ps curr 20 5 0x005F7FFC 0x005F7FC4 8192
sh $ uptime
inu has been up 1 second(s)

ew Syscall is easy

ew Syscall is easy

sh $
```

TRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0

Pada tugas ini dilakukan pembuatan system call baru pada sistem operasi Xinu sesuai dengan panduan pada modul bagian 9.5, di mana system call merupakan mekanisme penting yang berfungsi sebagai penghubung antara program user dengan kernel untuk meminta layanan tertentu seperti manajemen proses atau output ke layar. Proses pembuatan diawali dengan membuat sebuah fungsi baru bertipe syscall pada direktori /xinu/system/, misalnya fungsi cetakpesan() yang berisi perintah kprintf untuk menampilkan teks ke console Xinu sebagai bentuk output dari kernel. Selanjutnya, agar fungsi tersebut dapat dikenali oleh seluruh bagian sistem, dilakukan penambahan prototype pada file /xinu/include/prototypes.h. Setelah itu, file sumber yang berisi syscall tersebut harus didaftarkan ke dalam Makefile pada direktori system agar ikut diproses saat kompilasi. Tahap berikutnya adalah memanggil fungsi syscall tersebut di dalam file main.c sehingga ketika sistem dijalankan, syscall akan dieksekusi secara otomatis. Setelah semua konfigurasi selesai, dilakukan proses kompilasi dengan perintah make clean dan make pada direktori /xinu/compile untuk memastikan seluruh perubahan terbangun dengan benar tanpa konflik dari build sebelumnya. Kemudian sistem dijalankan menggunakan sudo minicom untuk melihat hasil eksekusi secara langsung. Berdasarkan hasil pengujian, output yang telah diprogram berhasil ditampilkan di layar, yang menunjukkan bahwa system call yang dibuat telah berjalan dengan baik tanpa error baik saat kompilasi maupun saat runtime. Hal ini membuktikan bahwa kernel Xinu bersifat modular dan dapat dengan mudah dikembangkan dengan menambahkan layanan baru melalui mekanisme system call, sehingga memberikan fleksibilitas bagi pengguna dalam mengembangkan fitur sesuai kebutuhan sistem.

```
#include <xinu.h>
```

```
/* chname.c - chname */
```

```
pri16 chname(pid32 pid, pri16 newprio)
```

```
{
```

```
    intmask mask;
```

```
    struct procent *prptr;
```

```
    pri16 oldprio;
```

```
    mask = disable();
```

```
    if (isbadpid(pid)) {
```

```
        restore(mask);
```

```
        return (pri16) SYSERR;
```

```
    }
```

```
    prptr = &proctab[pid];
```

```
    oldprio = prptr->prprio;
```

```
    prptr->prprio = newprio;
```

```
    kprintf("\n\nNew Syscall is easy\n\n");
```

```
    restore(mask);
```

```
    return oldprio;
```

```
}
```

2.

Kode di atas merupakan implementasi system call `chprio` pada Xinu yang berfungsi untuk mengubah prioritas suatu proses berdasarkan `pid` yang diberikan, di mana fungsi ini menerima dua parameter yaitu `pid` sebagai identitas proses dan `newprio` sebagai nilai prioritas baru yang akan diberikan. Pada awal eksekusi, sistem menyimpan status interrupt ke dalam variabel `mask` menggunakan fungsi `disable()` untuk mencegah gangguan selama proses perubahan data penting berlangsung. Selanjutnya dilakukan validasi terhadap `pid` menggunakan `isbadpid(pid)` untuk memastikan bahwa proses yang dimaksud benar-benar ada, serta pengecekan bahwa nilai prioritas baru tidak boleh kurang dari atau sama dengan nol, karena prioritas harus bernilai positif; jika salah satu kondisi tersebut tidak terpenuhi maka sistem akan mengembalikan status error (`YSERR`) setelah mengembalikan kondisi interrupt dengan `restore(mask)`. Jika validasi berhasil, pointer `prptr` diarahkan ke entri tabel proses (`proctab`) sesuai `pid`, kemudian nilai prioritas lama disimpan ke dalam variabel `oldprio` sebelum diganti dengan nilai `newprio`. Setelah proses perubahan selesai, interrupt dikembalikan ke kondisi semula menggunakan `restore(mask)` untuk menjaga kestabilan sistem, dan fungsi mengembalikan nilai prioritas lama dengan operasi `0xffff & oldprio` untuk memastikan hasil yang dikembalikan berada dalam format yang benar. Secara keseluruhan, system call ini menunjukkan bagaimana kernel Xinu mengelola perubahan atribut proses secara aman, terkontrol, dan tervalidasi sehingga tidak mengganggu eksekusi sistem secara keseluruhan.

```
Activities gedit Mon 27 Apr, 07:21
chprio.c
(~/.xinu/system)
Save

*/
syscall chprio(
    pid32 pid, /* ID of process to change */
    pri16 newprio /* New priority */
)
{
    intmask mask; /* Saved interrupt mask */
    struct procent *prptr; /* Ptr to process's table entry */
    pri16 oldprio; /* Priority to return */

    mask = disable();
    if (isbadpid(pid)) {
        restore(mask);
        return SYSERR;
    }

    if (newprio <= 0) {
        restore(mask);
        return SYSERR;
    }

    prptr = &proctab[pid];
    oldprio = prptr->prprio;
    prptr->prprio = newprio;
    restore(mask);
    return 0xffff & oldprio;
}
```

C Tab Width: 8 Ln 16, Col 75 INS

/* chprio.c - chprio */

#include <xinu.h>

```
syscall chprio(
    pid32 pid,
    pri16 newprio
)
{
    intmask mask;
    struct procent *prptr;
    pri16 oldprio;

    mask = disable();

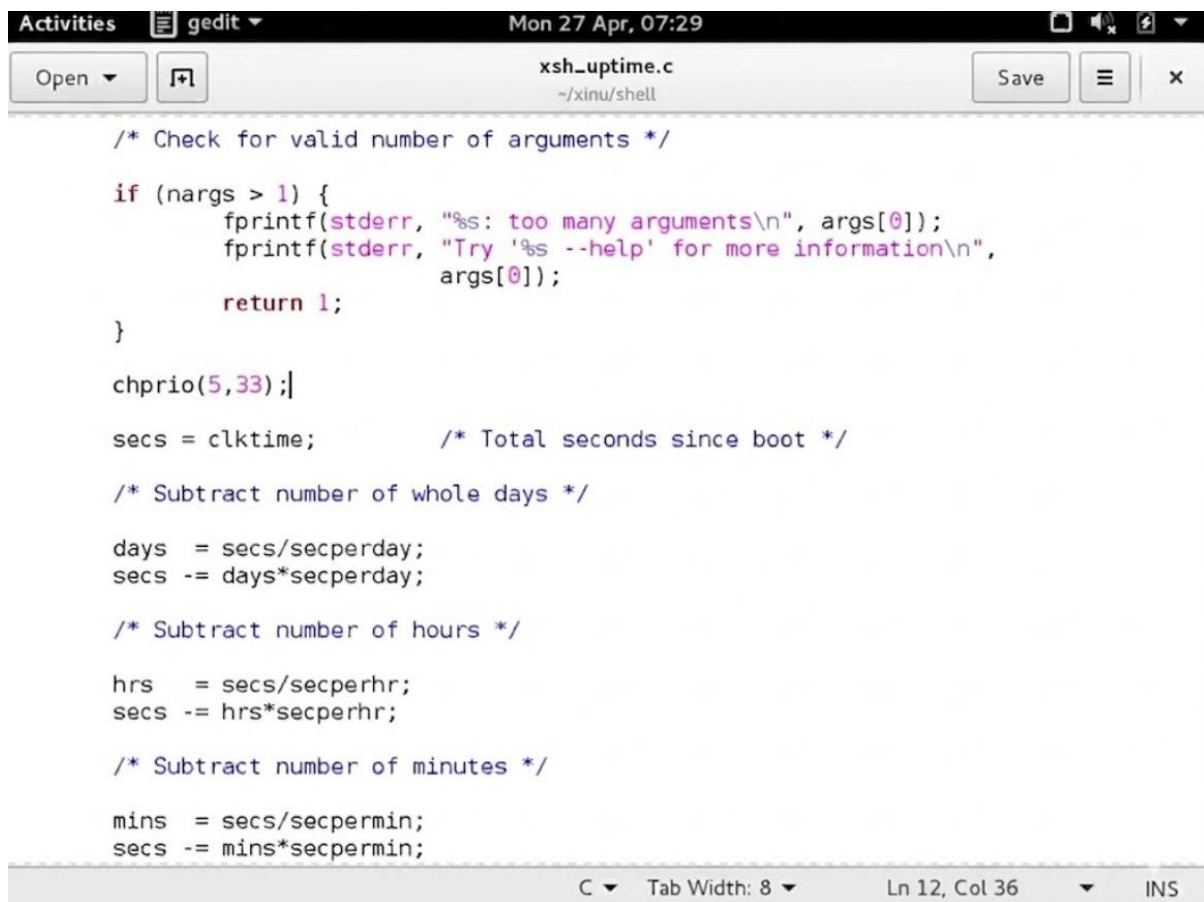
    if (isbadpid(pid)) {
        restore(mask);
        return SYSERR;
    }
}
```

```
if (newprio <= 0) {  
    restore(mask);  
    return SYSERR;  
}
```

```
prptr = &proctab[pid];  
oldprio = prptr->prprio;  
prptr->prprio = newprio;
```

```
restore(mask);  
return 0xffff & oldprio;  
}
```

3.



```
/* Check for valid number of arguments */

if (nargs > 1) {
    fprintf(stderr, "%s: too many arguments\n", args[0]);
    fprintf(stderr, "Try '%s --help' for more information\n",
            args[0]);
    return 1;
}

chprio(5,33);|

secs = clktime;          /* Total seconds since boot */

/* Subtract number of whole days */

days = secs/secperday;
secs -= days*secperday;

/* Subtract number of hours */

hrs = secs/secperhr;
secs -= hrs*secperhr;

/* Subtract number of minutes */

mins = secs/secpermin;
secs -= mins*secpermin;
```

C Tab Width: 8 Ln 12, Col 36 INS


```
Activities Terminal Mon 27 Apr, 08:48
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Help

obtained IP address 10.0.3.15 (0x0a00030f)

-----
XINU
-----

Welcome to Xinu!

sh $ ps
id Name State Prio Ppid Stack Base Stack Ptr Stack Size
-----
0 prnull ready 0 0 0x005FDFFC 0x00FFFE4 8192
1 ipout wait 500 0 0x005FBFFC 0x005FBF30 8192
2 netin wait 500 0 0x005F9FFC 0x005F9ED0 8192
4 Main process recv 20 3 0x005E7FFC 0x005E7F44 65536
5 shell recv 50 4 0x005D7FFC 0x005D79AC 8192
6 ps curr 20 5 0x005F7FFC 0x005F7FC4 8192
sh $

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

```

Activities  Terminal  Mon 27 Apr, 07:31
xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help
-----
0 prnull      ready    0      0 0x005FDFFC 0x00FFFE4 8192
1 ipout       wait     500    0 0x005FBFFC 0x005FBF30 8192
2 netin       wait     500    0 0x005F9FFC 0x005F9ED0 8192
4 Main process recv     20    3 0x005E7FFC 0x005E7F44 65536
5 shell       recv     50    4 0x005D7FFC 0x005D79AC 8192
6 ps          curr     20    5 0x005F7FFC 0x005F7FC4 8192
sh $ uptime

ew Syscall is easy

inu has been up 3 second(s)

ew Syscall is easy

sh $ ps
id Name          State Prio Ppid Stack Base Stack Ptr Stack Size
-----
0 prnull      ready    0      0 0x005FDFFC 0x00FFFE4 8192
1 ipout       wait     20      0 0x005FBFFC 0x005FBF30 8192
2 netin       wait     500     0 0x005F9FFC 0x005F9ED0 8192
4 Main process recv     20     3 0x005E7FFC 0x005E7F44 65536
5 shell       recv     33     4 0x005D7FFC 0x005D79AC 8192
8 ps          curr     20     5 0x005F7FFC 0x005F7FC4 8192
sh $
TRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0

```

```
/* Check for valid number of arguments */
```

```
if (nargs > 1) {
```

```
    fprintf(stderr, "%s: too many arguments\n", args[0]);
```

```
    fprintf(stderr, "Try %s --help for more information\n", args[0]);
```

```
    return 1;
```

```
}
```

```
/* Tambahan syscall */
```

```
chprio(5,33);
```

```
#include <xinu.h>
```

```
shellcmd xsh_uptime(int nargs, char *args[])
```

```
{
```

```
    int32 secs, mins, hrs, days;
```

```
    /* Check for valid number of arguments */
```

```
    if (nargs > 1) {
```

```

    fprintf(stderr, "%s: too many arguments\n", args[0]);
    fprintf(stderr, "Try %s --help for more information\n", args[0]);
    return 1;
}

/* Tambahan syscall */
chprio(5,33);

secs = clktime; /* Total seconds since boot */

days = secs / 86400;
secs -= days * 86400;

hrs = secs / 3600;
secs -= hrs * 3600;

mins = secs / 60;
secs -= mins * 60;

printf("Xinu has been up %d day(s), %d hour(s), %d minute(s), %d second(s)\n",
       days, hrs, mins, secs);

return 0;
}

```

Pada percobaan ini dilakukan modifikasi pada file xsh_uptime.c dengan menambahkan pemanggilan syscall `chprio(5,33)` setelah proses pengecekan jumlah argumen, yang bertujuan untuk mengubah prioritas suatu proses secara langsung ketika perintah `uptime` dijalankan. Secara konsep, syscall `chprio` digunakan untuk mengganti nilai prioritas dari proses yang memiliki Process ID (PID) tertentu, di mana pada kasus ini PID yang dituju adalah 5 (biasanya merupakan proses shell) dan prioritasnya diubah menjadi 33. Penempatan kode ini setelah validasi argumen penting agar fungsi tetap berjalan normal tanpa error sebelum syscall dipanggil. Ketika sistem dikompilasi ulang dan dijalankan, perintah `ps` pertama digunakan untuk melihat kondisi awal prioritas proses, kemudian setelah menjalankan perintah `uptime`, syscall `chprio` akan dieksekusi sehingga prioritas proses dengan PID 5 berubah. Perubahan ini kemudian dapat dibuktikan dengan menjalankan kembali perintah `ps`, di mana nilai prioritas proses tersebut akan terlihat meningkat menjadi 33. Dengan demikian, percobaan ini menunjukkan bahwa syscall yang ditambahkan berhasil diintegrasikan ke dalam sistem dan dapat memodifikasi atribut proses secara dinamis saat

runtime, sekaligus membuktikan pemahaman tentang mekanisme sistem call dan manajemen proses pada sistem operasi Xinu.